

K-Nearest Neighbors (KNN) :

- K-Nearest Neighbors (KNN) is a simple, supervised machine learning algorithm used for both classification and regression.
- It predicts the label of a new data point by finding the K most similar (closest) data points in the training set and selecting the majority class or averaging their values.
- It is a "lazy learner" because it stores data rather than building a model.

```
In [2]: import numpy as np
import pandas as pd
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score

df= pd.read_csv('Churn.csv')

# Check for null values in DataSet
#print(df.isna().sum())

# Fill the null values with the mean of that column
df.TotalCharges=df.TotalCharges.fillna(df.TotalCharges.mean())

X = df.drop(['customerID', 'Churn'], axis=1)
y = df['Churn']

# One-hot encode categorical variables
X = pd.get_dummies(X)

# Splitting the dataset for training and testing
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_stat

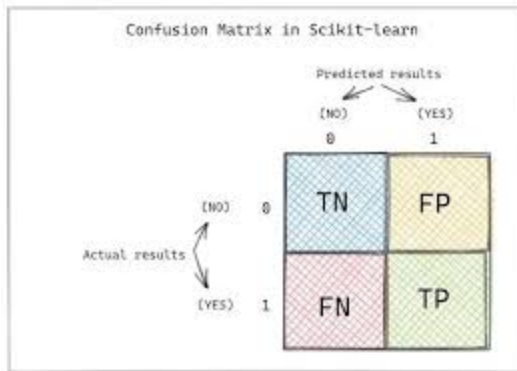
model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)
ypred = model.predict(X_test)
print(ypred)

['No' 'No' 'No' ... 'No' 'No' 'No']
```

Confusion Matrix

A confusion matrix is a performance evaluation tool for machine learning classification models, summarizing correct and incorrect predictions using a contingency table. It compares predicted classes against actual target values, highlighting specific errors made by

the model (e.g., mislabeling) rather than just overall accuracy.



Accuracy score:

Accuracy score in machine learning is a fundamental metric for classification models that calculates the proportion of correct predictions (both positive and negative) out of all predictions made.

Precision:

Precision is the ratio between the True Positives and all the Positives. It shows how many of the "yes" predictions made by the model were actually correct.

Recall:

Recall tells us how well a model finds all the correct "yes" cases in the data. It checks how many real positive cases the model was able to correctly identify.

- **Sensitivity (Recall / True Positive Rate):** Calculates the proportion of actual positive cases correctly identified.
 - *Formula:* $\frac{TP}{TP+FN}$ (True Positives / Actual Positives)
 - *Goal:* High sensitivity means few False Negatives (critical in disease detection).
- **Specificity (True Negative Rate):** Calculates the proportion of actual negative cases correctly identified.
 - *Formula:* $\frac{TN}{TN+FP}$ (True Negatives / Actual Negatives)
 - *Goal:* High specificity means few False Positives (critical in reducing false alarms). [GeeksforGeeks +4](#)

F1 score:

The F1 score is a machine learning evaluation metric that represents the harmonic mean of precision and recall, providing a balanced measure of a classification model's performance, particularly with imbalanced datasets.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$F_1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

```
In [3]: from sklearn.metrics import confusion_matrix, f1_score, classification_report
from sklearn.metrics import accuracy_score, precision_score, recall_score
```

```
cm=confusion_matrix(y_test, ypred)
print(cm)

accuracy = accuracy_score(y_test, ypred)
print("Accuracy of KNN: ", accuracy)

rs = recall_score(y_test, ypred, pos_label="Yes")
print("Recall Score: ", rs)

ps = precision_score(y_test, ypred, pos_label="Yes")
print("Precision Score: ", ps)

fs = f1_score(y_test, ypred, pos_label="Yes")
print("F1 Score: ", fs)

rep = classification_report(y_test, ypred)
print(rep)
```

```
[[938 123]
 [201 147]]
Accuracy of KNN: 0.7700496806245565
Recall Score: 0.4224137931034483
Precision Score: 0.5444444444444444
F1 Score: 0.47572815533980584
```

	precision	recall	f1-score	support
No	0.82	0.88	0.85	1061
Yes	0.54	0.42	0.48	348
accuracy			0.77	1409
macro avg	0.68	0.65	0.66	1409
weighted avg	0.75	0.77	0.76	1409

```
In [3]: import numpy as np
import pandas as pd
```

```

from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.datasets import load_iris

# Load the dataset
iris = load_iris()
df = pd.DataFrame(data=iris.data, columns=iris.feature_names)
df['target'] = iris.target
# Access the data
X = iris.data          # Features (sepal/petal measurements)
Y = iris.target        # Target Labels (0, 1, 2)
#feature_names = iris.feature_names
#target_names = iris.target_names

X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.2, random_state=42)

model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)
ypred = model.predict(X_test)

accuracy = accuracy_score(y_test, ypred)
print(accuracy)
cm=confusion_matrix(y_test, ypred)
print(cm)

```

0.9

```

[[10  0  0]
 [ 0 11  1]
 [ 0  2  6]]

```

Decision Tree Classifier

DecisionTreeClassifier with entropy is a Decision Tree algorithm used for **classification problems**, where the splitting criterion is based on **Entropy** (information gain).

Simple Meaning:

It creates a tree-like model by repeatedly splitting the dataset into groups so that each group becomes more **pure** (contains mostly one class).

What is Entropy?

Entropy measures the **impurity** or randomness in the data.

- **Entropy = 0** → Pure node (all records belong to one class)
- **Higher Entropy** → Mixed classes

Formula:

$$Entropy(S) = - \sum p_i \log_2(p_i)$$

Where:

- p_i = probability of each class

How It Works:

The algorithm checks all features and chooses the split that gives the **highest Information Gain**.

$$Information\ Gain = Entropy(parent) - Weighted\ Entropy(children)$$

```
In [5]: import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

df= pd.read_csv('Churn.csv')

# Check for null values in DataSet
print(df.isna().sum())

# Fill the null values with the mean of that column
df.TotalCharges=df.TotalCharges.fillna(df.TotalCharges.mean())

X = df.drop(['customerID', 'Churn'], axis=1)
y = df['Churn']

# One-hot encode categorical variables
X = pd.get_dummies(X)

# Splitting the dataset for training and testing
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = DecisionTreeClassifier(criterion="entropy", random_state=42, max_depth=2)
model.fit(X_train, y_train)
ypred = model.predict(X_test)

accuracy = accuracy_score(y_test, ypred)
print("Accuracy of Decision Tree: ",accuracy)
```

Accuracy of Decision Tree: 0.7601135557132718

```
In [6]: from sklearn import tree
print(tree.export_text(model))
```

```
|--- feature_36 <= 0.50
|   |--- feature_2 <= 93.67
|   |   |--- class: No
|   |--- feature_2 > 93.67
|   |   |--- class: No
|--- feature_36 > 0.50
|   |--- feature_16 <= 0.50
|   |   |--- class: No
|   |--- feature_16 > 0.50
|   |   |--- class: Yes
```

Random Forest Classifier

- A Random Forest Classifier is a supervised machine learning algorithm that builds an "ensemble" of many individual decision trees and combines their results to make a single, more accurate prediction.
- By aggregating the outputs of multiple trees, it reduces the risk of overfitting—a common problem where a single model learns the noise in a dataset rather than the general pattern.

```
model = RandomForestClassifier(n_estimators=100,  
criterion="entropy", random_state=42, max_depth=2)
```

Explanation of Each Parameter

1. `n_estimators=100`

Number of decision trees in the forest.

- Here, **100 trees** will be created.
- More trees usually improve accuracy but increase training time.

Example:

$$\textit{Total Trees} = 100$$

2. `criterion="entropy"`

Method used to split nodes in each decision tree.

Uses **Entropy / Information Gain** to choose the best split.

$$\textit{Entropy}(S) = - \sum p_i \log_2(p_i)$$

Lower entropy = better pure split.

3. `random_state=42`

Fixes randomness so results stay the same every time you run code.

Without it:

- Different trees may form each run.



With `42` :

- Reproducible results.

4. max_depth=2

Maximum depth of each tree = **2 levels**

Means tree can split only a little.

Example Tree:

Python

```
Root
├── Left Child
└── Right Child
    ├── Leaf
    └── Leaf
```

Benefits:

- Prevents overfitting
- Faster model
- Simpler trees



```
In [6]: import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

df= pd.read_csv('Churn.csv')

# Check for null values in DataSet
#print(df.isna().sum())

# Fill the null values with the mean of that column
df.TotalCharges=df.TotalCharges.fillna(df.TotalCharges.mean())

X = df.drop(['customerID', 'Churn'], axis=1)
y = df['Churn']

# One-hot encode categorical variables
X = pd.get_dummies(X)

# Splitting the dataset for training and testing
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = RandomForestClassifier(n_estimators=100, criterion="entropy", random_state=42)
model.fit(X_train, y_train)
ypred = model.predict(X_test)

accuracy = accuracy_score(y_test, ypred)
print("Accuracy of Random Forest: ",accuracy)
```

Accuracy of Random Forest: 0.7530163236337828

SVM

Support Vector Machine (SVM) is a powerful, supervised machine learning algorithm used primarily for classification and regression tasks. It works by finding the optimal hyperplane (decision boundary) that maximizes the margin (distance) between different data classes in an N-dimensional space, ensuring high-accuracy separation.

Key Aspects of SVM:

- **Optimal Hyperplane:** SVM seeks the best separating line (in 2D) or plane (in higher dimensions) that offers the largest separation between classes.
- **Support Vectors:** These are the data points closest to the hyperplane, which "support" or define the position and orientation of the margin.
- **Margin Maximization:** The goal is to maximize the distance between the support vectors of different classes to reduce generalization errors.
- **Kernel Trick:** SVM uses "kernel functions" to handle non-linear data by mapping input data into higher-dimensional spaces where it can be linearly separated.  IBM +6

Common Applications:

- **Image Recognition:** Used for face detection and object classification.
- **Text Classification:** Used for spam detection and sentiment analysis.
- **Bioinformatics:** Applied to classify protein sequences or cancer tissues.  Great Learning +4

```
In [8]: import pandas as pd
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

df= pd.read_csv('Churn.csv')

# Fill the null values with the mean of that column
df=df.fillna(df.mean(numeric_only=True))

X = df.drop(['customerID', 'Churn'], axis=1)
y = df['Churn']

# One-hot encode categorical variables
X = pd.get_dummies(X)

# Splitting the dataset for training and testing
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_stat

model=SVC(C=10, kernel='rbf')
model.fit(X_train, y_train)
```

```
ypred = model.predict(X_test)

accuracy = accuracy_score(y_test, ypred)
print("Accuracy of SVM: ", accuracy)
```

Accuracy of SVM: 0.7892122072391767

In []:

In []: